Ain Shams University - Faculty of Engineering

Digital Design Verification Course - Final Project

Spring 2026

Grading Interface Contract

SPI Master Verification Project - Spring 2026

Rev 1.2 - Spring 2026

This document is normative.

Any student submission that violates a MUST clause forfeits the corresponding rubric item. Sections marked 'informative' are non-binding guidance; the Makefile interface (Section 2) and the log contract (Section 4) are the canonical grader hooks.

This file is **normative**. Any student submission that violates a **MUST** clause in this file forfeits the corresponding rubric item.

1. File-system contract

Your submission zip must unpack to a single top-level directory named `<name>_<id>/` that contains at minimum:

```
<name>_<id>/
  tb/                    # testbench top-level file(s)
  env/                   # agents, scoreboard, coverage, reference model
  tests/                 # test classes / programs
  sequences/             # stimulus sequences
  assertions/            # SVA bind files (or inline in env/)
  docs/
    test_plan.pdf
    final_report.pdf
    coverage_report.pdf  # exported after `make cov`
  Makefile               # MUST inherit the targets below
  README.md              # how to run, toolchain notes
```

Additional folders are allowed but are ignored by the grader.

The directory layout is mandatory regardless of methodology. An SV-only submission populates the *same* folders but with SystemVerilog programs and modules instead of UVM class libraries (see Section 9).

2. Required Makefile targets (MUST)

Your `Makefile` MUST support exactly the following invocations, with no additional positional arguments required:

Target	Effect
`make compile`	Compile the testbench and the DUT sources given by `DUT_SRCS`
`make run TEST=<name> SEED=<n>`	Run a single test with the given seed
`make regress`	Run the full regression list, once per `REGRESSION_SEEDS`
`make run_bonus`	Run the mandatory RAL bonus test `ral_hw_reset_test`
`make cov`	Produce `coverage_report.txt` at the repo root
`make clean`	Remove build artefacts

The grader will invoke:

```
make clean
make compile DUT_SRCS="<abs_path_to_spi_core_buggy> \
                       <abs_path_to_apb_regfile_buggy> \
                       <abs_path_to_spi_master_buggy>"
make regress DUT_SRCS="..." REGRESSION_SEEDS=<N>
make cov
```

`REGRESSION_SEEDS` is bounded such that **total runs <= 10000**.

DUT_SRCS vs DUT_SRC

The DUT now has three RTL files (`spi_core.sv`, `apb_regfile.sv`, `spi_master.sv`) that together implement the IP. The grader passes the whole list via `DUT_SRCS`.

For backward compatibility the grader's older single-file override `DUT_SRC=<one file>` is still honoured by the template Makefile: if `DUT_SRC` is non-empty it replaces `DUT_SRCS` completely. Students MUST NOT set `DUT_SRC` themselves.

run_bonus

`make run_bonus` runs a test whose name is fixed to `ral_hw_reset_test` (i.e. the Makefile variable `BONUS_TEST` defaults to this). Students who do not attempt the RAL bonus SHOULD keep a stub test under that name that prints `[TEST_SKIPPED] ral_hw_reset_test`; the grader's RAL detector then counts zero for the bonus and the rest of the rubric is unaffected.

3. Log message contract (MUST)

The grader recognises a bug catch **if and only if** at least one of the following appears in the test log, on its own line (leading whitespace allowed):

- `[SCOREBOARD_ERROR] <free text>` - mismatch in predicted vs observed
- `[ASSERTION_ERROR] <assertion name> <free text>` - SVA failure
- `[CHECKER_ERROR] <check name> <free text>` - explicit `$error` from a check
- Any standard Questa/VCS `** Error:` or `UVM_FATAL` / `UVM_ERROR` line from your own code is also accepted.

At the end of each test, your TB MUST print exactly one of:

- `[TEST_PASSED] <test_name>` - zero errors, all expected events seen
- `[TEST_FAILED] <test_name> errors=<n>` - at least one error

The grader uses these lines to count passes/fails independently from the bug-detection count.

4. Coverage contract (MUST)

- `coverage_report.txt` must be produced by `make cov` after a full regression
- The report MUST include total functional and code coverage numbers
- The grader parses the last occurrence of lines of the form `Total Coverage: <pct>%` and `Functional Coverage: <pct>%`
- Coverage gate: functional >= 85% and code (statement/branch) >= 85% on the **golden** RTL. This gate is checked in a separate grader pass using `DUT_SRC=<path to golden>`

5. Runtime contract (MUST)

- `make regress` must finish in **at most 10000 simulation invocations**. Total runs = `|REGRESSION_TESTS| x REGRESSION_SEEDS`.
- A single test must finish in at most 10 minutes of wall time on the grading machine (reasonable modern 8-core). Timeouts are treated as test failures.
- **Per-bug regression budget (NEW)**: when the grader injects one specific seeded bug and runs `make regress` to detect it, the **entire regression MUST complete in <= 5 minutes of wall time** on the grading machine. Practically this means:
  - Sum of all per-test wall times across all seeds, for the regression list you declare in `REGRESSION_TESTS`, must be `<= 300 s`.
  - Compile time (`make compile`) is **not** counted toward the 5 min; only `make regress` after compile is measured.
  - The grader will start a 5 min timer right before `make regress` and kill the simulation if it exceeds it. Tests still running at the timeout are scored as `[TEST_FAILED]` and any bug they would have caught is counted as missed.
  - This budget is per bug. The grader runs one bug per regression pass and re-invokes `make regress` for the next bug; the 5 min budget restarts each time.
- Tests must be deterministic given `+SEED=<n>` - the grader re-runs failing tests once to verify.

Note
Practical guidance: budget your regression so the product `|REGRESSION_TESTS| x REGRESSION_SEEDS x avg_test_seconds <= 300 s`. A common shape is ~10 tests x ~5 seeds x ~6 s per run. Tests that naively burn many SCLK periods at large `DIV` values MUST cap `DIV` or shorten the transfer count to fit the budget.

5a. Simulation flow (informative)

This section documents the exact command sequence the grader uses so students can reproduce a grading pass locally. It is informative, not normative - the normative interface is the Makefile target list in Section 2.

End-to-end flow per bug:

```
                +-----------------------+
                | grader picks bug_i    |
                | (DUT_SRCS = buggy)    |
                +----------+------------+
                           |
                           v
                +-----------------------+
                | make clean            |   wipes work/, build/, *.ucdb
                +----------+------------+
                           |
                           v
                +-----------------------+
                | make compile          |   vlog: harness + DUT + TB
                | DUT_SRCS=<buggy>      |   coverage instrumented
                +----------+------------+
                           |
                           v
                +-----------------------+   <-- 5 min wall-time budget
                | make regress          |       starts here
                | REGRESSION_SEEDS=N    |   loops:
                |                       |     for t in REGRESSION_TESTS:
                |                       |       for s in 1..N:
                |                       |         vsim ... +TESTNAME=t +SEED=s
                |                       |         > build/log_t_s.log
                |                       |   merges cov_*.ucdb
                +----------+------------+
                           |
                           v
                +-----------------------+
                | make cov              |   vcover report
                |                       |   -> coverage_report.txt
                +----------+------------+
                           |
                           v
                +-----------------------+
                | grader scans logs:    |
                |   [SCOREBOARD_ERROR]  |
                |   [ASSERTION_ERROR]   |
                |   UVM_ERROR/UVM_FATAL |
                | bug_i caught? (yes/no)|
                +-----------------------+
```

To reproduce a single test locally exactly the way the grader does:

```
make clean
make compile DUT_SRCS="<core> <regfile> <top>"
make run TEST=sanity_test SEED=1 WAVES=1     # WAVES=1 dumps a .wlf
```

To reproduce the full regression with the 5 min budget:

```
make clean
make compile DUT_SRCS="<core> <regfile> <top>"
time make regress REGRESSION_SEEDS=10        # MUST print real <= 5m00s
make cov
```

The grader's bug-detection pass differs from the above only in that `DUT_SRCS` points at a `*_buggy.sv` file instead of the golden RTL.

6. UVM bonus contract (OPTIONAL, strictly enforced)

If you attempt the UVM bonus, the grader runs **structural** checks on your source tree. All of the following MUST be observed for the UVM bonus to apply:

1. `tb/tb_top.sv` (or any file under `tb/`) contains `import uvm_pkg::*;` AND calls `run_test()`; the default test name passed to `run_test` MUST be one of your own `uvm_test` subclasses. 2. At least one class in `env/` extends `uvm_env` and defines a `build_phase` that calls `uvm_config_db#(virtual <iface>)::get(...)` at least once for the APB virtual interface. 3. At least one test in `tests/` registers a `uvm_factory` type override via

`set_type_override_by_type` (or `set_inst_override_by_type`) at build time. Comment-only overrides do not count. 4. A `uvm_subscriber`-derived (or `uvm_analysis_imp`-based) coverage collector exists in `env/` and is instantiated somewhere inside the `uvm_env`.

If UVM RAL is used, you additionally qualify for the RAL bonus if ALL of the following hold:

1. A class in `env/` extends `uvm_reg_block` and declares nine `uvm_reg` handles whose names match the register map in the spec (`CTRL`, `STATUS`, `TX_DATA`, `RX_DATA`, `CLK_DIV`, `SS_CTRL`, `INT_EN`, `INT_STAT`, `DELAY`). 2. An `add_hdl_path`/`configure`/`add_hdl_path_slice` call wires the backdoor paths to `dut_wrapper.u_dut.u_regfile.*` (the exposed DUT hierarchy). Backdoor access MUST be demonstrated at runtime - a single `UVM_BACKDOOR` read/write is sufficient. 3. `tests/ral_hw_reset_test.sv` (exactly that filename, class name `ral_hw_reset_test extends uvm_test`) runs `uvm_reg_hw_reset_seq` against the register block and prints `[TEST_PASSED] ral_hw_reset_test` on the golden RTL. 4. `make run_bonus` succeeds on the golden RTL.

The grader performs these checks via:

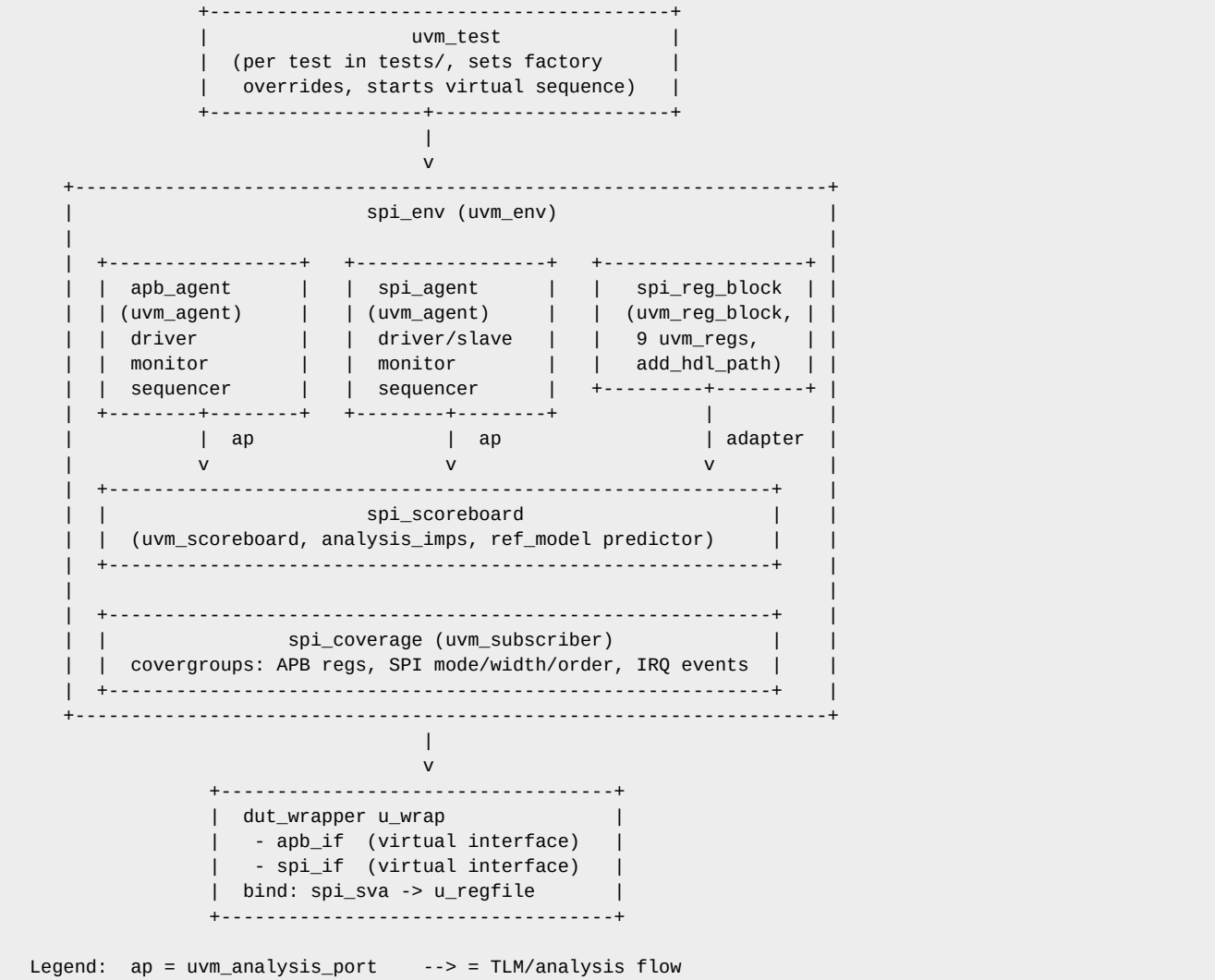
- Regex/AST search across `tb/`, `env/`, `tests/`.
- An actual `make run_bonus DUT_SRCS=<golden>` invocation. The RAL bonus is awarded **only** if that run prints `[TEST_PASSED] ral_hw_reset_test`.

Partial attempts (e.g. a `uvm_reg_block` declared but never used, no backdoor, or `ral_hw_reset_test` missing) do **not** earn the RAL bonus. They may still earn the UVM bonus if the UVM structural checks pass.

6a. UVM environment layout (informative)

If you opt into the UVM bonus, the recommended environment is shown below. This section is **informative**: any structure that satisfies Section 6 is accepted. The diagram exists so reviewers and graders share a mental model of "what a healthy UVM env for this DUT looks like".

High-level block diagram



Required UVM components

Component	UVM base	Lives in	Notes
APB transaction	`uvm_sequence_item`	`env/`	randomisable addr/data/write/strb fields
APB driver	`uvm_driver`	`env/`	drives `apb_if.master` modport
APB monitor	`uvm_monitor`	`env/`	publishes observed APB beats on its `analysis_port`
APB agent	`uvm_agent`	`env/`	active by default; UVM_PASSIVE supported via config
SPI transaction	`uvm_sequence_item`	`env/`	mode, width, MSB/LSB-first, MISO byte pattern
SPI driver (slave-side)	`uvm_driver`	`env/`	drives `spi_if.slave` (MISO + tracks SCLK/SS_n)
SPI monitor	`uvm_monitor`	`env/`	reconstructs MOSI bytes per transfer
SPI agent	`uvm_agent`	`env/`	one instance per slave; configurable via config_db
Reference model	`uvm_component`	`env/`	predicts RX/IRQ/STATUS from APB+SPI traffic per spec
Scoreboard	`uvm_scoreboard`	`env/`	analysis_imps from monitors + ref_model; emits `[SCOREBOARD_ERROR]`
Coverage collector	`uvm_subscriber`	`env/`	covergroups for APB ops, SPI mode/width/order, IRQ events
Register block (RAL bonus)	`uvm_reg_block`	`env/`	9 `uvm_reg` handles + `add_hdl_path` to `dut_wrapper.u_dut.u_regfile`
Sequencer(s)	`uvm_sequencer`	`env/` (in agents)	one per agent, plus an optional virtual sequencer in `env`
Sequences	`uvm_sequence`	`sequences/`	reusable stimulus (single transfer, mode sweep, FIFO stress, ...)
Tests	`uvm_test`	`tests/`	one per scenario you cover; sets factory overrides; starts seqs

Recommended folder layout for a UVM submission

```
<name>_<id>/
  tb/
    tb_top.sv                     # module: clk/rst, ifaces, dut_wrapper, run_test()
  env/
    spi_pkg.sv                    # `package` import-all: agents+env+sb+cov+ref_model
    apb_agent_pkg.sv              # APB seq_item, driver, monitor, agent
    spi_agent_pkg.sv              # SPI seq_item, slave driver, monitor, agent
    ref_model.sv                  # spec predictor (uvm_component)
    scoreboard.sv                 # uvm_scoreboard with analysis_imps
    coverage.sv                   # uvm_subscriber with covergroups
    spi_reg_block.sv              # uvm_reg_block (RAL bonus only)
    spi_env.sv                    # uvm_env: builds + connects everything
  sequences/
    spi_seq_pkg.sv                # base + library sequences (mode_sweep, fifo_stress, ...)
  tests/
    spi_base_test.sv              # uvm_test: builds env, common phase config
    sanity_test.sv                # student-defined tests, each extends spi_base_test
    <your_test_2>.sv              # add as many as you need to hit coverage and bugs
    <your_test_3>.sv
    ...
    ral_hw_reset_test.sv          # RAL bonus only
  assertions/
    spi_sva.sv                    # bound into u_wrap.u_dut.u_regfile (and/or u_core)
  docs/
    test_plan.pdf
    final_report.pdf
    coverage_report.pdf
  Makefile
  README.md
```

Recommended Makefile shape for a UVM submission

The UVM Makefile is the SV-only template with two changes: an extra `+UVM_NO_RELNOTES` flag at run time and the UVM package on the compile line. Sketch:

```
SIMULATOR ?= questa
SEED      ?= 0
TEST      ?= sanity_test
WAVES     ?= 0

PROJ_ROOT  ?= ../..
HARNESS    ?= $(PROJ_ROOT)/harness

# DUT sources (overridden by grader to point at faulty_rtl/*_buggy.sv)
DUT_SRCS   ?= \
  $(PROJ_ROOT)/golden_rtl/spi_core.sv \
  $(PROJ_ROOT)/golden_rtl/apb_regfile.sv \
  $(PROJ_ROOT)/golden_rtl/spi_master.sv

# Student sources, in compile order (env before sequences before tests)
ENV_SRCS    = env/spi_pkg.sv
SEQ_SRCS    = sequences/spi_seq_pkg.sv
ASSERT_SRCS = assertions/spi_sva.sv
TEST_SRCS   = tests/spi_base_test.sv tests/sanity_test.sv ... tests/ral_hw_reset_test.sv
TB_SRCS     = tb/tb_top.sv

INC_DIRS    = +incdir+$(HARNESS) +incdir+./env +incdir+./sequences +incdir+./tests +incdir+./tb

# Regression list - student-defined. The grader runs every test name
# listed here. There are no required test names; size the list to fit
# the 5 min per-bug regression budget (Section 5).
REGRESSION_TESTS =                      # fill in your own tests
REGRESSION_SEEDS ?= 5                   # tune jointly with REGRESSION_TESTS

VLOG_FLAGS = -sv -timescale=1ns/1ps +acc=rn +define+SIM $(INC_DIRS) -L mtiUvm
COV_FLAG   = -coverage +cover=bcestf

compile:
    vlib work
    vlog $(VLOG_FLAGS) $(COV_FLAG) \
       $(HARNESS)/apb_if.sv $(HARNESS)/spi_if.sv \
       $(DUT_SRCS) $(HARNESS)/dut_wrapper.sv \
       $(ENV_SRCS) $(SEQ_SRCS) $(ASSERT_SRCS) $(TEST_SRCS) $(TB_SRCS)

run: compile
    vsim -c -L mtiUvm work.tb_top \
         -do "run -all; coverage save cov_$(TEST)_$(SEED).ucdb; quit -f" \
         +UVM_TESTNAME=$(TEST) +UVM_NO_RELNOTES +SEED=$(SEED) \
         $(if $(filter 1,$(WAVES)),-wlf waves_$(TEST)_$(SEED).wlf,)

run_bonus: compile
    vsim -c -L mtiUvm work.tb_top -do "run -all; quit -f" \
         +UVM_TESTNAME=ral_hw_reset_test +UVM_NO_RELNOTES +SEED=$(SEED)

regress: compile
    @for t in $(REGRESSION_TESTS); do \
       for s in `seq 1 $(REGRESSION_SEEDS)`; do \
         $(MAKE) -s run TEST=$$t SEED=$$s WAVES=0 > build/log_$${t}_$${s}.log 2>&1 ; \
       done ; \
     done
    -vcover merge -out build/merged.ucdb $(wildcard cov_*.ucdb)

cov:
    vcover report -details build/merged.ucdb > coverage_report.txt

clean:
    rm -rf work build *.wlf *.ucdb transcript coverage_report.txt
```

Key contract points for the UVM track (already in Section 6, repeated here as a cross-reference):

- `tb/tb_top.sv` MUST `import uvm_pkg::*;` and call `run_test()`.
- `spi_env` MUST `uvm_config_db#(virtual apb_if)::get(...)` (and the same for `spi_if`) in its `build_phase`.
- At least one `uvm_test` MUST register a factory type override at build time.
- A `uvm_subscriber`-derived coverage collector MUST be instantiated inside `spi_env`.
- For RAL: `spi_reg_block` MUST declare the nine `uvm_reg` handles named exactly as in the spec, wire

backdoor paths to `dut_wrapper.u_dut.u_regfile.*`, and `tests/ral_hw_reset_test.sv` MUST run `uvm_reg_hw_reset_seq` and print `[TEST_PASSED] ral_hw_reset_test`.

7. Forbidden practices

- Hardcoding the DUT source path (must use `DUT_SRC`)
- Short-circuiting tests based on `+define+FAULTY` or similar: the grader will compile with `+define+SIM` only
- Detecting the grader by hostname/username/env var
- Consuming more than 10 GB of disk or RAM during a single test
- Calling external services at test time

Violations void the bug-detection score.

8. Submission checklist

- [ ] Zip named `<name>_<id>.zip`
- [ ] `make compile` succeeds with the default `DUT_SRCS` (golden RTL)
- [ ] `make regress` finishes under the 10000-run cap
- [ ] `coverage_report.txt` produced and >= 85% on golden RTL
- [ ] `docs/test_plan.pdf`, `docs/final_report.pdf`, `docs/coverage_report.pdf` all present
- [ ] `README.md` lists simulator version, any non-default compile flags
- [ ] No binary simulator output files checked in (`*.wlf`, `work/`, etc.)
- [ ] If UVM bonus attempted: `make run_bonus` passes on golden and the RAL structural checks in Section 6 all hold.

9. SystemVerilog-only track (mandatory baseline)

UVM is **optional**. If you choose to stay in plain SystemVerilog, your submission MUST still fit the Section 1 layout. The grader does not treat SV-only and UVM submissions differently for the base rubric - only the UVM and RAL bonuses are off the table.

Recommended organisation for an SV-only submission:

```
tb/
  tb_top.sv              # module (not a class) instantiating dut_wrapper
  apb_master_bfm.sv      # `program` or `module` that drives apb_if
  spi_slave_bfm.sv       # module that drives/monitors spi_if.slave
env/
  ref_model.sv           # `module` or `program` implementing the predictor
  scoreboard.sv          # pure SV scoreboard (queues + $error on mismatch)
  coverage.sv            # covergroups on APB, SPI, interrupt events
tests/
  sanity_test.sv         # `program` automatic; selected by +TESTNAME=...
  reg_access_test.sv
  mode_coverage_test.sv
  ...
sequences/
  stim_lib.sv            # reusable randomisable transaction classes
assertions/
  spi_sva.sv             # module bound to dut_wrapper via `bind`
```

`tb/tb_top.sv` MUST look roughly like this skeleton (full version is shipped in `harness/examples/sv_only/`):

```
`timescale 1ns/1ps
module tb_top;
  // clk / rst / interfaces
  bit PCLK = 0; always #5 PCLK = ~PCLK;
  bit PRESETn;
  apb_if apb (PCLK, PRESETn);
  spi_if spi ();

  dut_wrapper u_wrap (.apb(apb.slave), .spi(spi), .PCLK(PCLK), .PRESETn(PRESETn));

  // bind assertions (use the *instance path* of your dut_wrapper instance,
  // here `u_wrap`, NOT the bare module-type `dut_wrapper`)
  bind u_wrap.u_dut.u_regfile spi_sva u_sva (.*);

  // test dispatch: +TESTNAME=<name> selects which program to spawn
  string testname;
  initial begin
    PRESETn = 0;
    #50 PRESETn = 1;
    if (!$value$plusargs("TESTNAME=%s", testname)) testname = "sanity_test";
    case (testname)
      "sanity_test"          : sanity_test         :: run(apb, spi);
      // add one arm per test you list in REGRESSION_TESTS
      default                : $fatal(1, "Unknown test %s", testname);
    endcase
    $display("[TEST_PASSED] %s", testname);
    $finish;
  end
endmodule
```

Key contract points for SV-only:

- The grader drives `+TESTNAME=<name>` AND `+UVM_TESTNAME=<name>`. Your TB MUST honour at least one of them; the template Makefile already passes both.
- Every test MUST end with exactly one `[TEST_PASSED]` or `[TEST_FAILED]` line as in Section 3.
- Scoreboards MUST signal mismatches via `[SCOREBOARD_ERROR] ...` lines (Section 3). `$error` is also accepted.
- SVA MUST live in a file that is bound into the DUT (e.g. via `bind u_wrap.u_dut.u_regfile ...` or at the `u_wrap.u_dut.u_core` level). Inline assertions inside the DUT are not counted because you cannot modify the DUT.
- `REGRESSION_TESTS` in the Makefile MUST list every test you want the grader to run; the list is student-defined and there are no required test names.

A ready-to-clone SV-only scaffold lives at `harness/examples/sv_only/` (shipped with the starter kit). Start from there if you are not using UVM.